

Организация параллельных программ как системы ПОТОКОВ

Процессы и потоки

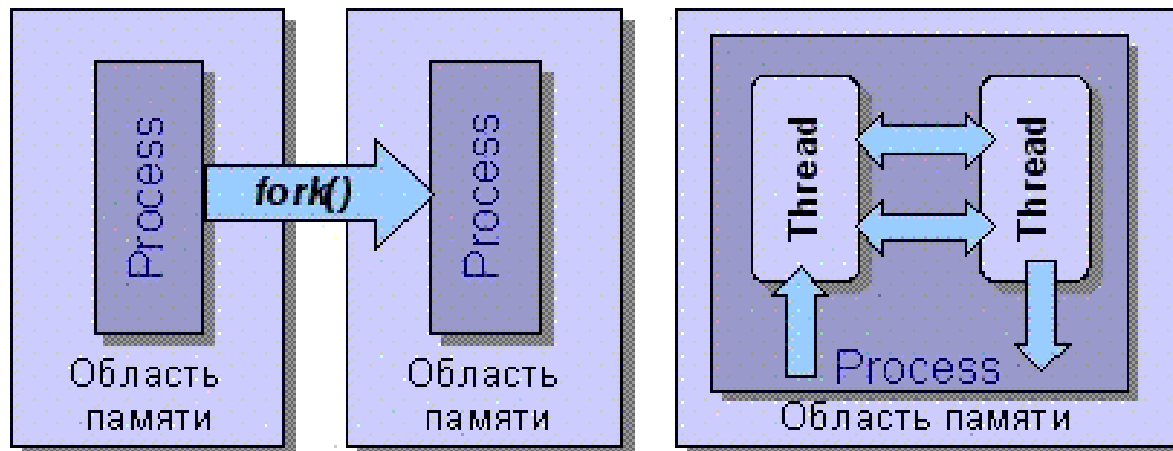
- **Процесс** – объект, который создается Операционной Системой для приложения (не для приложения .NET) в момент его запуска. Характеризуется **собственным адресным пространством**, которое напрямую недоступно другим процессам.
- В рамках процесса создаются **потоки**. Это последовательность выполняемых команд процессора. В приложении может быть несколько потоков (первичный поток и дополнительные потоки).

- Потоки в процессе разделяют совместно используемые данные и имеют собственные стеки вызовов и локальную память потока (Thread Local Storage – TLS).

TLS потока содержит информацию:

- о ресурсах, используемых потоком,
- о регистрах,
- состоянии памяти,
- выполняемой инструкции процессора.
- В приложении все потоки выполняются в рамках **общего адресного пространства**.

- Чем отличаются два процесса и два потока, выполняющих одну и ту же задачу?



Домен приложения

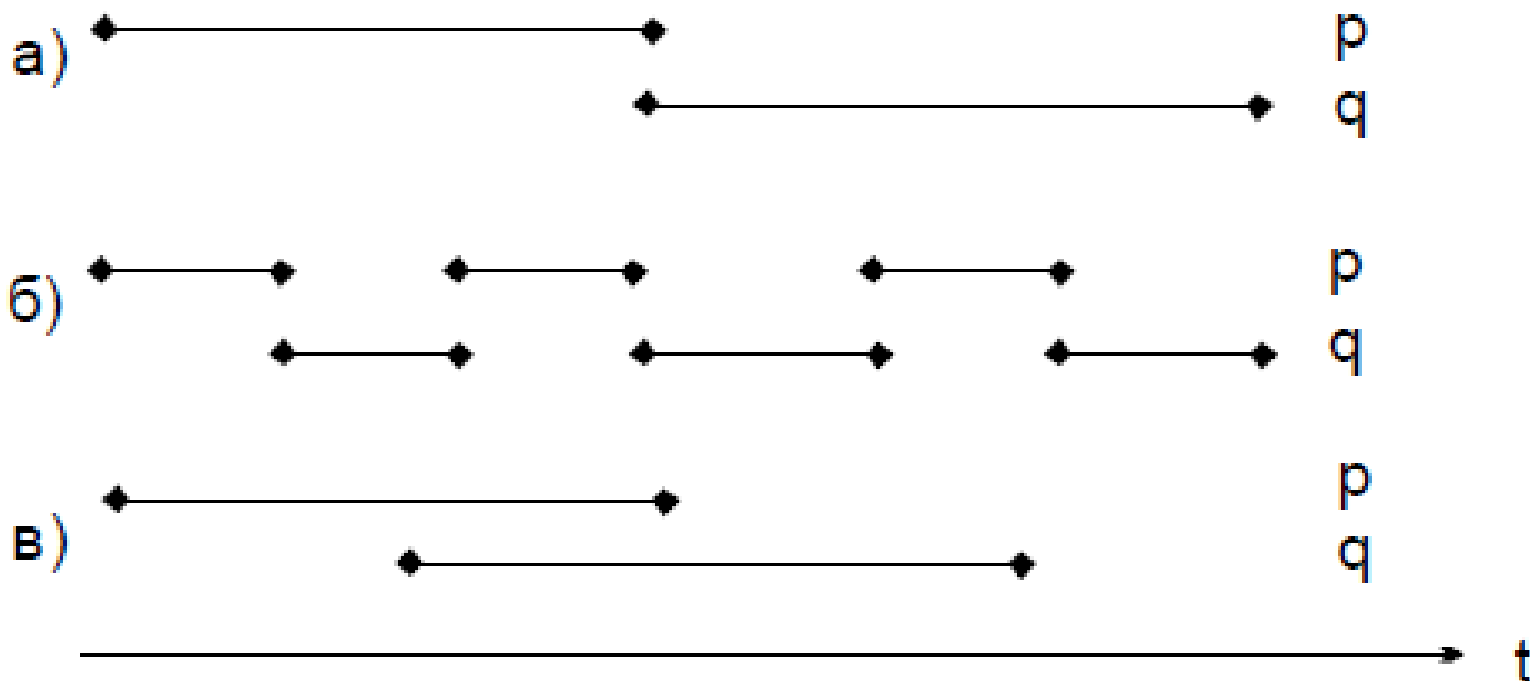
- Выполнение приложений .NET начинается с запуска .NET Framework. Это процесс со своими потоками, специальными атрибутами и правилами взаимодействия с другими процессами.
- Приложения .NET выполняются в **ОДНОМ процессе**. Для этих приложений процесс .NET Framework играет роль, аналогичную роли операционной системы при обеспечении выполнения процессов.
- Выполняемые в процессе .NET Framework .NET-приложения также изолируются друг от друга. Средством изоляции .NET-приложений являются **домены приложений**.

Домен приложения

- Домен приложения изолирует (на логическом уровне) выполняемые приложения и используемые в его рамках ресурсы.
- В процессе .NET Framework может выполняться множества доменов приложений. При этом в рамках одного домена может выполняться множество потоков.
- Функциональность и основные свойства домена приложения реализуются в классе **System.AppDomain**.

- Понятие потока может быть использовано в качестве основного *конструктивного элемента* для построения параллельных программ в виде совокупности ***взаимодействующих потоков***.
- Такая агрегация программы позволяет получить более компактные (поддающиеся анализу) вычислительные схемы реализуемых методов, скрыть при выборе способов распараллеливания несущественные детали программной реализации, обеспечивает концентрацию усилий на решение основных проблем параллельного функционирования программ.

Варианты взаиморасположения траекторий одновременно исполняемых потоков



А - однопрограммный режим работы ЭВМ.

Б - выполнение потоков может осуществляться одновременно, но в каждый момент времени могут исполняться команды только какого либо одного потока (режим разделения времени или многопрограммный режим работы ЭВМ),

В - параллельное выполнение потоков, когда одновременно могут выполняться команды нескольких потоков (данный режим исполнения потоков осуществим только при наличии в вычислительной системе нескольких процессоров).

Состояния потока

- В условиях многопоточного приложения разные потоки могут переводить друг друга в разные состояния. Таким образом, поток может находиться одновременно **более чем в одном** состоянии.
- Например, если поток блокирован в результате вызова метода `Wait`, а другой поток вызвал по отношению к блокированному потоку метод `Abort`, то блокированный поток окажется в одно и то же время в состояниях `WaitSleepJoin` и `AbortRequested`.

Таблица допустимых состояний потока

[illegible]

Взаимодействие и взаимоисключение потоков

Одной из причин зависимости результатов выполнения программ от порядка чередования команд может быть разделение одних и тех же данных между одновременно исполняемыми потоками. Данная ситуация может рассматриваться как проявление общей **проблемы использования разделяемых ресурсов** (общих данных, файлов, устройств и т.п.). Для организации разделения ресурсов между несколькими потоками необходимо иметь возможность:

- определения доступности запрашиваемых ресурсов (ресурс свободен и может быть выделен для использования, ресурс уже занят одним из потоков программы и не может использоваться дополнительно каким-либо другим потоком);
- выделения свободного ресурса одному из потоков, запросивших ресурс для использования;
- приостановки (**блокировки**) потоков, выдавших запросы на ресурсы, занятые другими потоками.

- Главное требование к механизмам разделения ресурсов является гарантированное обеспечение **использования каждого разделяемого ресурса только одним потоком** от момента выделения ресурса этому потоку до момента освобождения ресурса.
- Данное требование в литературе обычно именуется *взаимоисключением потоков (mutual exclusion)*; командные последовательности потоков, в ходе которых поток использует ресурс на условиях взаимного исключения, называется **критической секцией** потока.
- С использованием последнего понятия условие взаимного исключения потоков может быть сформулировано как требование **нахождения в критических секциях по использованию одного и того же разделяемого ресурса не более чем одного потока**.

Вариант 1. Жесткая синхронизация

```
int ThreadNum=1; // номер потока для доступа к ресурсу
Thread_1() {
while (1) {
// повторять, пока право доступа к ресурсу у потока 2
while ( ThreadNum == 2 );
<Использование общего ресурса>
// передача права доступа к ресурсу потоку 2
ThreadNum = 2;
}
}
Thread_2() {
while (1) {
// повторять, пока право доступа к ресурсу у потока 1
while ( ThreadNum == 1 );
< Использование общего ресурса >
// передача права доступа к ресурсу потоку 1
ThreadNum = 1;
}
}
```

Недостатки жесткой синхронизации

- ресурс используется потоками строго последовательно (по очереди) и, как результат, при разном темпе развития потоков общая скорость выполнения программы будет определяться наиболее медленным потоком;
- при завершении работы какого-либо потока другой поток не сможет воспользоваться ресурсом и может оказаться в постоянно заблокированном состоянии.

Вариант 2. Потеря взаимoisключения

```
int ResourceThread1 = 0; // =1 - ресурс занят потоком 1
int ResourceThread2 = 0; // =1 - ресурс занят потоком 2
Thread_1() {
while (1) {
// повторять, пока ресурс используется потоком 2
while ( ResourceThread2 == 1 );
ResourceThread1 = 1;
< Использование общего ресурса >
ResourceThread1 = 0;
}
}
Thread_2() {
while (1) {
// повторять, пока ресурс используется потоком 1
while ( ResourceThread1 == 1 );
ResourceThread2 = 1;
< Использование общего ресурса >
ResourceThread2 = 0;
}
}
```

- В отдельных случаях взаимоисключение потоков в данном примере может произойти корректно - все определяется конкретными моментами переключения потоков.
- Успешность однократного выполнения не может служить доказательством правильности функционирования параллельной программы даже при неизменных параметрах решаемой задачи.
- Для выявления ошибочных ситуаций необходима проверка разных временных траекторий выполнения параллельных потоков.

Вариант 3. Возможность взаимоблокировки

```
int ResourceThread1 = 0; // =1 - ресурс занят потоком 1
int ResourceThread2 = 0; // =1 - ресурс занят потоком 2

Thread_1() {
while (1) {
// установить, что поток 1 пытается занять ресурс
ResourceThread1 = 1;
// повторять, пока ресурс занят потоком 2
while ( ResourceThread2 == 1 );
< Использование общего ресурса >
ResourceThread1 = 0;
}}

Thread_2() {
while (1) {
// установить, что поток 2 пытается занять ресурс
ResourceThread2 = 1;
// повторять, пока ресурс используется потоком 1
while ( ResourceThread1 == 1 );
< Использование общего ресурса >
ResourceThread2 = 0;
}}
```

- Возникает новая проблема – оба потока могут оказаться заблокированными вследствие бесконечного повторения циклов ожидания освобождения ресурсов (что происходит при одновременной установке управляющих переменных в состояние "занято").
- Данная проблема известна под названием ситуации **тупика (дедлока (deadlock) или смертельного объятия)** и исключение тупиков является одной из наиболее важных задач в теории и практике параллельных вычислений.

Вариант 4. Бесконечное откладывание

```
int ResourceThread1 = 0; // =1 - ресурс занят потоком 1
int ResourceThread2 = 0; // =1 - ресурс занят потоком 2
Thread_1() {
while (1) {
ResourceThread1=1; // поток 1 пытается занять ресурс
// повторять, пока ресурс занят потоком 2
while ( ResourceThread2 == 1 ) {
ResourceThread1 = 0; // снятие занятости ресурса
<временная задержка>
ResourceThread1 = 1;
}
< Использование общего ресурса >
ResourceThread1 = 0;
}
}
```

```
Thread_2() {  
while (1) {  
ResourceThread2=1; // поток 2 пытается занять ресурс  
// повторять, пока ресурс используется потоком 1  
while ( ResourceThread1 == 1 )  
{  
ResourceThread2 = 0; // снятие занятости ресурса  
<временная задержка>  
ResourceThread2 = 1;  
}  
< Использование общего ресурса >  
ResourceThread2 = 0;  
}  
}
```

- Длительность временной задержки в циклах ожидания должна определяться при помощи некоторого случайного датчика.
- При таких условиях реализованный алгоритм обеспечивает взаимоисключение и исключает возникновение тупиков, но опять таки не лишен существенного недостатка.
- Проблема состоит в том, что потенциально решение вопроса о выделении может откладываться до бесконечности (при синхронном выполнении потоков).
- Данная ситуация известна под наименованием **бесконечное откладывание (starvation)** или **длительной блокировкой (live lock)**.

Алгоритм Деккера

```
int ThreadNum=1; // номер потока для доступа к ресурсу
int ResourceThread1 = 0; // =1 – ресурс занят потоком 1
int ResourceThread2 = 0; // =1 – ресурс занят потоком 2
Thread_1() {
while (1) {
ResourceThread1=1; // поток 1 пытается занять ресурс
// цикл ожидания доступа к ресурсу
while ( ResourceThread2 == 1 ) {
if ( ThreadNum == 2 ) {
ResourceThread1 = 0;
// повторять, пока ресурс занят потоком 2
while ( ThreadNum == 2 );
ResourceThread1 = 1;
}}
< Использование общего ресурса >
ThreadNum = 2;
ResourceThread1 = 0;
}}
```

Алгоритм Деккера (продолжение)

```
Thread_2() {  
while (1) {  
ResourceThread2=1; // поток 2 пытается занять  
ресурс  
// цикл ожидания доступа к ресурсу  
while ( ResourceThread1 == 1 ) {  
if ( ThreadNum == 1 ) {  
ResourceThread2 = 0;  
// повторять, пока ресурс используется потоком 1  
while ( ThreadNum == 1 );  
ResourceThread2 = 1;  
}}  
< Использование общего ресурса >  
ThreadNum = 1;  
ResourceThread2 = 0;  
}}
```

- Алгоритм Деккера гарантирует корректное решение проблемы взаимного исключения для двух потоков.
- Управляющие переменные *ResourceThread1*, *ResourceThread2* обеспечивают взаимное исключение, переменная *ThreadNum* исключает возможность бесконечного откладывания. Если оба потока пытаются получить доступ к ресурсу, то поток, номер которого указан в *ThreadNum*, продолжает проверку возможности доступа к ресурсу (внешний цикл ожидания ресурса).
- Другой же поток в этом случае снимает свой запрос на ресурс, ожидает своей очереди доступа к ресурсу (внутренний цикл ожидания) и возобновляет свой запрос на ресурс.

- Алгоритм Деккера может быть обобщен на случай произвольного количества потоков, однако такое обобщение приводит к заметному усложнению выполняемых действий.
- Кроме того, программное решение проблемы взаимного исключения потоков приводит к нерациональному использованию процессорного времени ЭВМ (потоку, ожидающему освобождения ресурса, постоянно требуется процессор для проверки возможности продолжения – *активное ожидание (busy wait)*).

Семафоры

- Под *семафором* S понимается переменная особого типа, значение которой может опрашиваться и изменяться только при помощи специальных операций $P(S)$ и $V(S)$.
- Принципиальным в понимании семафоров является то, что операции $P(S)$ и $V(S)$ предполагаются неделимыми (атомарными), что гарантирует взаимоисключение при использовании общих семафоров (для обеспечения неделимости операции обслуживания семафоров обычно реализуются средствами операционной системы).
- Различают два основных типа семафоров. **Двоичные семафоры** принимают только значения 0 и 1, область значений **общих семафоров** – неотрицательные целые значения.
- В момент создания семафоры инициализируются некоторым целым значением.

Операции над семафором

- операция **$P(S)$**

если $S > 0$ то $S = S - 1$

иначе <ожидать S >

- операция **$V(S)$**

если <один или несколько потоков ожидают S >

то <снять ожидание у одного из ожидающих потоков>

иначе $S = S + 1$

Операции над семафором

- **P-операция** над семафором представляет собой попытку декремента значения семафора на 1. Если перед выполнением P-операции значение семафора было больше 0, то P-операция выполняется без задержек. Если перед выполнением P-операции значение семафора было равным 0, то процесс, выполняющий P-операцию, переводится в состояние ожидания до тех пор, пока значение семафора не станет большим 0.
- **V-операция** над семафором представляет инкремент значения семафора на 1. Если при этом имеются процессы, задержанные на выполнении P-операции на данном семафоре, один из этих процессов выходит из состояния ожидания и может выполнить свою P-операцию.

Взаимное исключение на семафоре

- Для реализации взаимного исключения, например, предотвращения возможности одновременного изменения двумя или более процессами общих данных, создается двоичный семафор S .
- Начальное значение этого семафора = 1.
- Критические секции кода обрамляются «скобками» $P(S)$ (в начале секции) и $V(S)$ (в конце секции).
- Процесс, входящий в критическую секцию, выполняет операцию $P(S)$ и переводит семафор в 0.
- Если в критической секции уже находится другой процесс, то значение семафора уже 0, тогда второй процесс, желающий войти в критическую секцию, блокируется в своей P -операции до тех пор, пока процесс, находящийся сейчас в критической секции, не выйдет из нее, выполнив на выходе операцию $V(S)$.

```
Semaphore Sem=1; // семафор взаимногоисключения потоков
Thread_1() {
while (1) {
// проверить семафор и ждать, если ресурс занят
P(Sem) ;
< Использование общего ресурса >
// освободить один из ожидающих ресурса потоков
// увеличить семафор, если нет ожидающих потоков
V(Sem) ;
}}
Thread_2() {
while (1) {
// проверить семафор и ждать, если ресурс занят
P(Sem) ;
< Использование общего ресурса >
// освободить один из ожидающих ресурса потоков
// увеличить семафор, если нет ожидающих потоков
V(Sem) ;
}}
```

- На практике в разных средах выполнения наряду с поддержкой "стандартных" семафоров реализуются некоторые их разновидности – *мьютексы (mutex)*, *замки (lock)* и др.
- Вариации касаются допустимого набора значений для переменных семафоров, рекурсивности вызова, введения дополнительного набора операций и т.д.

Мониторы

- *Мониторы* представляют собой программные модули (объекты), которые реализуют все необходимые действия с разделяемым ресурсом.
- Общий формат определения монитора может быть представлен следующим образом:

```
Monitor <Name> {  
    <объявления переменных>  
    <операторы инициализации>  
    <процедуры монитора>  
}
```


- Как можно заметить, описание монитора достаточно близко совпадает с описанием класса в C++.
- Принципиальное отличие состоит в том, что процедуры монитора, в обязательном порядке, выполняются *в режиме взаимного исключения*, т.е. при выполнении какой-либо процедуры монитора все остальные попытки вызова других процедур этого же монитора блокируются.
- Обеспечение такого правила выполнения процедур монитора должно осуществляться средой выполнения, в которой поддерживается концепция мониторов.

Пример

```
Monitor SharedMem {
```

```
int N=0; // Общая переменная
```

```
procedure Set (int v) { //Операция записи
```

```
N = v;
```

```
}
```

```
procedure Get (int &v) { //Операция чтения
```

```
v = N;
```

```
}
```

```
procedure Inc () { //Операция изменения
```

```
N = N + 1;
```

```
}
```

```
}
```

Свойства монитора:

- Переменные монитора недоступны вне монитора и могут обрабатываться только процедурами монитора;
- Вне монитора доступны только процедуры монитора;
- Переменные, объявленные вне монитора, недоступны внутри монитора.